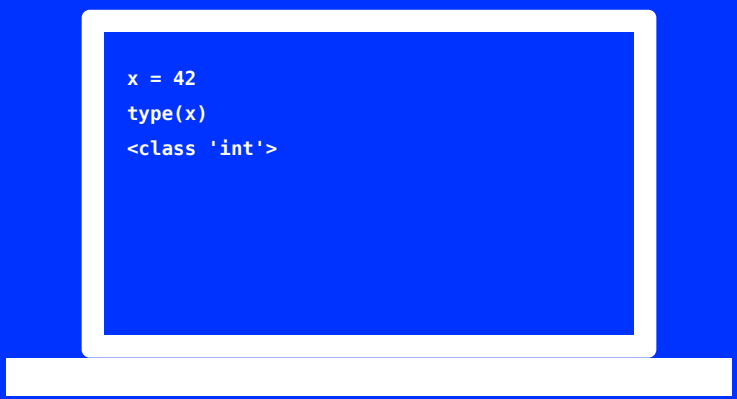
{ PISCINE IA }

DAY 02

< VARIABLES & OPERATORS />

Week 1 - Python for AI



v 1.0

DAY 02

Today's goal

Variables, data types and operators are the vocabulary of every AI program. A text becomes a `str`, a score a `float`, a token id an `int`, an answer a `bool`.

Activity	Time	Today
Course and understanding	40 min	Variables, types, operators
Practical exercises	45 min	Tasks 01 to 03
Programming	60 min	Tasks 04 and onwards
AI project (Legal-Benin)	30 min	Legal angle of the day
Review and documentation	15 min	Self-check + push

i Tip: Type every example yourself in the REPL before putting it in a file. Predict the result **before** pressing Enter.

Task 01 - Types

Write `task01/types.py`. For each value below, print its type exactly as in the example.

✓ `42`, `3.14`, `"Article 12"`, `True`, `None`, `[1, 2, 3]`, `{"id": 7}`, `(1, 2)`

Terminal

```
$> python3 types.py
42 -> <class 'int'>
3.14 -> <class 'float'>
...
```

i Tip: `type(x)` gives the type, `isinstance(x, int)` tests it. Note that `True` is also an `int` in Python. Explain why in `SELFCHECK.md`.

Task 02 - calc.py

Write `task02/calc.py` which takes three command line arguments: a number, an operator (`+`, `-`, `*`, `/`, `//`, `%`, `**`) and another number, and prints the result.

Terminal

```
$> python3 calc.py 7 // 2
3
$> python3 calc.py 7 / 2
3.5
$> python3 calc.py 7 / 0
Error: division by zero
$> python3 calc.py 7
Usage: calc.py <a> <op> <b>
```

i Tip: Arguments are in `sys.argv`, always as strings. Convert them, and decide what to do when the conversion fails.

⚠ **Warning:** No `eval()`. It executes arbitrary code and is forbidden in the piscine.

Task 03 - Strings, the raw material of NLP

Write `task03/article_info.py`. Given this line stored in a variable:

input

```
line = "Article 12 : Tout citoyen a le droit de saisir la justice."
```

Print, using f-strings and slicing (no loops, no regex yet):

- ✓ the article number as an `int`,
- ✓ the text after the colon, stripped,
- ✓ the number of characters and the number of words of that text,
- ✓ the text in UPPERCASE and its first 10 characters.

Terminal

```
$> python3 article_info.py
number: 12
text: Tout citoyen a le droit de saisir la justice.
chars: 45, words: 9
upper: TOUT CITOYEN A LE DROIT DE SAISIR LA JUSTICE.
preview: Tout citoy
```

⚠ **Warning:** This sentence is a **fictional example** to practise on. Real articles from your corpus are for later.

Task 04 - Operators and precedence

Without running them, write your prediction of these results in `task04/predictions.md`, then run the code and correct yourself. Explain every surprise.

predictions

```
2 + 3 * 4 ** 2
10 / 4, 10 // 4, 10 % 4, -10 // 4
0.1 + 0.2 == 0.3
"ab" * 3 + "c"
not (3 > 2 and 5 < 4) or False
bool(""), bool("0"), bool([]), bool(0.0)
```

i Tip: `0.1 + 0.2` is the classic: floats are approximations. You will meet this again with gradients and losses. See `math.isclose`.

Task 05 - Mean and variance by hand

Without NumPy, in `task05/stats.py`, compute from the values below the mean, the variance and the standard deviation, using only variables and operators (a loop is allowed if you already know it).

input

```
scores = [12, 15, 9, 20, 14, 18]
```

Terminal

```
$> python3 stats.py
mean: 14.6667
variance: 13.2222
std: 3.6362
```

i Tip: Variance here is the **population** variance (divide by n). Check the result with the `statistics` module.

Task 06 - Mini report

Write `task06/report.py` that asks the user a name and an age with `input()`, then prints a message that tells if the person is a minor or an adult and how many years remain until 100. Handle a non numeric age without crashing (`try` / `except ValueError`).

Self-check (no notes!)

Rule 4 of the piscine: you must be able to explain your work without looking at your notes. Answer these out loud, then write one sentence each in **SELFCHECK.md**.

1. What is the difference between `/`, `//` and `%`?
2. Why is `0.1 + 0.2 != 0.3`?
3. Which type would you use for: a token id, a loss value, a yes/no label, an article text?

Delivery

Everything goes in your repository, in **week01/day02/**, with one commit per task (clear messages, in English). Run **python3 -m py_compile** on each script before pushing.

⚠ **Warning:** Never push your virtual environment (`.venv/`), notebooks with secrets, or huge data files. A `.gitignore` is part of your delivery.